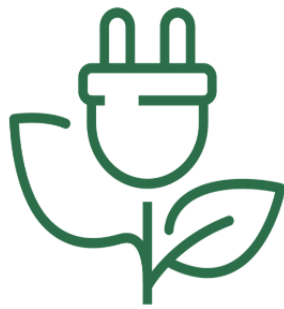




Università degli Studi di Trento

Dipartimento di Ingegneria e Scienza dell'Informazione



Rinova

Sviluppo

A cura degli studenti

Stefano Pezzetta

Enrico Sartori

Documento di sviluppo dell'applicazione

Anno accademico 2025/2026

Indice

1	Scopo del documento	2
2	Web APIs	3
2.1	Scelte di design delle API	3
2.2	Documentazione	3
2.3	Specifica OpenAPI	3
3	Implementation	6
3.1	Motivazioni dello Stack Tecnologico	6
3.2	Organizzazione repository	7
3.3	Branching Strategy e Organizzazione del lavoro	8
3.4	Dependencies	9
3.5	Database	10
3.6	Testing	11
4	FrontEnd	13
4.1	Descrizione FrontEnd	13
4.2	Navigazione	13
4.3	Logiche di Visualizzazione e Funzionalità	13
4.4	Schermate Principali	14
5	Deployment	19
5.1	Accesso all'Applicativo	19
5.2	Contatti Assistenza	20

1 Scopo del documento

Il presente documento riporta tutte le informazioni necessarie per lo sviluppo di alcuni dei servizi dell'applicazione Rinova. In particolare, presenta tutti gli artefatti necessari per realizzare i servizi di reportistica, visualizzazione e costruzione di grafici, dashboard ed interazione fra utenti.

Il documento parte dalla presentazione delle Web APIs, mostra l'organizzazione del codice ed il modello dati utilizzato, e fornisce alcune informazioni sulle modalità e tecnologie adottate per il testing. Infine, si conclude con una sezione dedicata al front-end e una agli aspetti di deployment.

2 Web APIs

2.1 Scelte di design delle API

Per lo sviluppo delle API abbiamo optato per l'utilizzo di **Python FastAPI**. Questa scelta è stata motivata dalla necessità di elaborare in modo efficiente i dati energetici e le serie storiche per la reportistica e la dashboard. Le API seguono i principi dell'architettura **RESTful**:

- Utilizzo standard dei metodi HTTP (es. GET per il recupero dei dati storici e live).
- Formato di scambio dati in JSON, facilitato dalla validazione automatica offerta da *Pydantic*.
- Gestione degli errori standardizzata tramite `HTTPException` (es. 422 per Validation Error, 500 per Internal Server Error).
- Autenticazione gestita tramite header `Authorization` (con token JWT forniti da Supabase).

2.2 Documentazione

Le API sono state documentate secondo le specifiche OpenAPI 3.1. La documentazione interattiva è consultabile pubblicamente tramite Swagger UI al seguente indirizzo:

<https://rinova-energy-api.onrender.com/docs>

o altrimenti tramite **Redocly** al seguente indirizzo:

<https://rinova-energy-api.onrender.com/redoc>

2.3 Specifica OpenAPI

La specifica completa delle API è disponibile nel repository Github del progetto. Il contenuto del file di specifica (convertito in formato `.yaml` per maggiore leggibilità) è qui riportato per esteso:

```
openapi: 3.1.0
info:
  title: Rinova Energy API
  description: API Backend per la gestione e il monitoraggio impianti fotovoltaici.
  version: 1.1.0
  license:
    name: MIT
servers:
  - url: https://rinova-api.onrender.com
    description: Production Server
```

```

- url: http://localhost:8000
  description: Local Development Server
paths:
  /:
    get:
      tags: [General]
      summary: Health Check
      responses:
        '200':
          description: Successful Response
  /api/dati-privati:
    get:
      tags: [General, Utils]
      summary: Leggi Dati Sensibili
      responses:
        '200':
          description: Successful Response
  /api/production/live:
    get:
      tags: [Dashboard]
      summary: Dati Live Dashboard
      responses:
        '200':
          description: Successful Response
  /api/dashboard/summary:
    get:
      tags: [Dashboard]
      summary: KPI Sommario Home
      responses:
        '200':
          description: Successful Response
  /api/dashboard/chart:
    get:
      tags: [Dashboard]
      summary: Grafico Home (4 Ore)
      responses:
        '200':

```

```

        description: Successful Response
/api/dashboard/plants:
  get:
    tags: [Dashboard]
    summary: Lista Widget Impianti
    responses:
      '200':
        description: Successful Response
/api/production/history:
  get:
    tags: [Analytics]
    summary: Storico Produzione
    parameters:
      - name: period
        in: query
        required: true
        description: 'Periodo: week, month, year, custom'
      - name: plantId
        in: query
        required: true
    responses:
      '200':
        description: Successful Response
/api/report/download:
  get:
    tags: [Reports]
    summary: Download Report PDF
    parameters:
      - name: period
        in: query
        required: true
      - name: plantId
        in: query
        required: true
    responses:
      '200':
        description: Successful Response

```

3 Implementation

L'applicazione Rinova è stata sviluppata discostandosi dallo stack tecnologico proposto dal corso in favore di un'architettura più moderna, distribuita e orientata all'analisi dei dati. Lo stack si articola in tre componenti principali:

- **Frontend (Client):** Vite + React + TypeScript per i grafici, le tabelle e la UI base della Progressive WebApp.
- **Middleware Computazionale / Data Analysis:** Python FastAPI, che agisce come microservizio specializzato per le elaborazioni complesse (calcolo KPI, manipolazione serie storiche e generazione report PDF), interconnettendo il database con le viste analitiche del frontend.
- **BaaS (Backend-as-a-Service):** Supabase (PostgreSQL), che funge da core per la persistenza dei dati, l'autenticazione (Auth) e le logiche di sicurezza (RLS).

3.1 Motivazioni dello Stack Tecnologico

Questa scelta architetturale è stata motivata da conoscenze pregresse del team e, soprattutto, da esigenze specifiche del dominio applicativo (dashboard energetiche) che richiedevano strumenti più robusti rispetto allo stack di base.

Perché Supabase (PostgreSQL) e non MongoDB?

La differenza fondamentale risiede nella natura relazionale dei dati trattati. Il dominio delle Comunità Energetiche (Utenti, Impianti, CER, dati di produzione) è fortemente interconnesso. PostgreSQL gestisce queste relazioni rigorose in modo nativo e robusto, laddove un database NoSQL a document-based come MongoDB risulterebbe meno efficiente e più prone a inconsistenze per aggregazioni complesse. Inoltre, l'ecosistema Supabase ci ha permesso di centralizzare e semplificare enormemente lo sviluppo grazie alle seguenti feature integrate:

- **Gestione dell'Autenticazione (Auth):** Supabase gestisce tutto "il lavoro sporco" relativo a sessioni, generazione, aggiornamento e validazione dei JWT. Si integra inoltre perfettamente con provider OAuth esterni (come Google e Azure Auth, utilizzati nella WebApp) fornendo una soluzione sicura e pronta all'uso.
- **Row Level Security (RLS):** Le policy di sicurezza a livello di riga sono native di PostgreSQL e gestite in modo molto intuitivo tramite l'interfaccia di Supabase. Questo ci ha permesso di blindare l'accesso ai dati alla radice (es. un utente può leggere solo i log dei propri impianti), senza dover implementare complessi controlli dal lato codice.

- **Intelligenza Artificiale (Supabase AI):** L'assistente AI integrato in Supabase è uno strumento prezioso per la manutenzione del database. Non legge i dati privati, ma assiste nella stesura delle policy RLS, suggerisce la creazione di indici per ottimizzare le performance delle query e analizza la struttura del database per segnalare eventuali vulnerabilità o difetti progettuali.
- **Ecosistema completo (Storage, Edge Functions, Cron, Realtime):** Oltre al database, Supabase fornisce un sistema di Storage per file/documenti facilmente implementabile, un'interfaccia API REST generata in automatico e un sistema di log integrato. Supporta inoltre funzionalità avanzate come *Edge Functions*, esecuzione di *Cron Jobs* per task pianificati e le sottoscrizioni in *Realtime* ai cambiamenti del database.

Perché Python FastAPI e React+TypeScript?

- **Elaborazione Analitica (FastAPI vs Node.js):** Python è lo standard industriale per l'analisi dei dati. FastAPI ci ha permesso di gestire calcoli matematici complessi (es. KPI energetici sulle serie storiche) e la generazione di report PDF personalizzati con prestazioni e librerie che un ambiente Node.js puro avrebbe reso molto più ostici da implementare.
- **Robustezza del Frontend (Vite+React+TS vs JS Vanilla):** TypeScript garantisce la *type-safety*, riducendo drasticamente i bug a runtime e agevolando lo sviluppo. React, unito alla velocità di build di Vite, è ideale per gestire in modo reattivo e dichiarativo lo stato complesso di una dashboard ricca di grafici, filtri e dati in tempo reale.

3.2 Organizzazione repository

Il codice del progetto (<https://github.com/plaspi/Rinova>) è organizzato secondo la seguente struttura:

```
.github/workflows      # Configurazione CI e test applicazione

/webapp                # Root directory dell'applicazione
  index.html           # File HTML base
  vite.config.ts       # Configurazione PWA (Manifest, Service Worker)
  .env.example         # File esempio configurazione variabili d'ambiente
  .gitignore           # Configurazione Repository git

/src                  # Cartella file sorgenti FrontEnd (React + TypeScript)
  /components          # Componenti UI (es. forms/, tables/, ui/, sidebar/)
  /context             # React Context per la gestione di stato (Auth, Plants)
```


/hooks	# Custom hooks (es. gestione mobile)
/lib	# Helper CSS
/pages	# Viste Pagine principali principali (homePage, CerPage, Plant
/services	# Configurazione API e client Supabase
/tests	# Configurazione Vitest
/backend	# Codice sorgente BackEnd (Python FastAPI)
/routers	# Definizione degli endpoint (analytics, dashboard...)
/services	# Logica di autenticazione e interazione con Supabase
/tests	# Suite di test Python (es. test_dashboard.py)
main.py	# Entry point dell'applicazione ASGI FastAPI
schemas.py	# Modelli Pydantic per validazione schemi e I/O
exceptions.py	# Gestione personalizzata degli errori
requirements.txt	# Dipendenze pip del backend
/D1	# Cartella contenente la documentazione (Deliverable 1)
/D2	# Cartella contenente la documentazione (Deliverable 2)
/D3	# Cartella contenente la documentazione (Deliverable 3)
/D4	# Cartella contenente la documentazione (Deliverable 4)

3.3 Branching Strategy e Organizzazione del lavoro

Lo sviluppo del progetto ha richiesto una pianificazione attenta e flessibile, specialmente a seguito di una variazione nella composizione iniziale del team (passato da tre a due membri in corso d'opera). Questo cambiamento ha richiesto una rapida redistribuzione dei carichi di lavoro. In totale, sul repository sono stati eseguiti oltre **70 commit**.

La collaborazione è stata gestita tramite incontri di allineamento settimanali, fondamentali per fare il punto della situazione, assegnare i task e coordinare le naturali fasi di stallo fisiologico dovute alla preparazione di altri esami universitari.

Per ottimizzare i tempi, la divisione dei compiti è stata nettamente specializzata in base alle attitudini dei membri:

- **Sviluppo Full-Stack (Enrico):** Si è fatto carico della quasi totalità dell'implementazione tecnica, curando l'architettura di base, il backend in FastAPI, lo sviluppo dell'interfaccia React e l'integrazione con Supabase.
- **Project Management e Documentazione (Stefano):** Ha assunto un ruolo di coordinamento e stesura documentale, curando in modo approfondito la redazione dei documenti Deliverables e

contribuendo alla rifinitura di specifici componenti e dettagli della UI.

A livello di repository, data la flessibilità richiesta dalla riorganizzazione del team e la necessità di uno sviluppo rapido, abbiamo optato per una strategia di branching pragmatica e orientata agli obiettivi didattici, allontanandoci da un rigido GitFlow in favore di un approccio più snello (simile al Trunk-Based Development).

Il flusso di lavoro si è basato su un branch di integrazione principale (es. dev o main), sul quale venivano integrati frequentemente i progressi. I branch aggiuntivi sono stati creati seguendo due direttrici principali:

- **Branch di Feature:** Creati all'occorrenza per sviluppare funzionalità specifiche o complessi refactoring (es. backAPI o fix particolari) senza rompere l'ambiente di sviluppo principale.
- **Branch di Documentazione / Deliverable:** Utilizzati per isolare la stesura dei documenti di progetto rispetto al codice sorgente. Questo ci ha permesso di lavorare in parallelo sulla corposa reportistica senza "sporcare" la cronologia del branch di sviluppo principale, in vista dell'unica consegna cumulativa finale del progetto.

3.4 Dependencies

Il progetto si basa su un ecosistema di librerie moderne e performanti. Di seguito le dipendenze principali utilizzate:

Frontend (npm):

- **Core e Stato:** react e react-dom per l'interfaccia, react-router-dom per la navigazione e jotai per il global state management (leggero e basato su atomi).
- **Forms e Validazione:** react-hook-form combinato con zod per la gestione performante dei moduli e la validazione rigorosa dei dati lato client.
- **Stile e UI Components:** tailwindcss per lo styling utility-first, combinato con i componenti accessibili di Shadcn UI. Per i feedback visivi (toast) si è optato per sonner, mentre per le icone lucide-react.
- **Visualizzazione Dati:** recharts per il rendering reattivo dei grafici energetici e leaflet (con react-leaflet) per la geolocalizzazione e visualizzazione degli impianti su mappa.
- **BaaS e Tooling:** @supabase/supabase-js per l'interazione con il database. Il bundling e la configurazione della PWA (Manifest, Service Workers) sono gestiti tramite vite e vite-plugin-pwa.

Backend (pip):

- **Core API e Validazione:** fastapi per la stesura degli endpoint REST, serviti tramite uvicorn. L'I/O e la serializzazione sono tipizzati rigorosamente tramite pydantic.
- **Elaborazione Analitica:** pandas per l'aggregazione avanzata delle serie storiche di produzione/consumo energetico.
- **Reportistica:** matplotlib per la generazione statica di grafici backend e fpdf per la compilazione procedurale dei report PDF scaricabili dall'utente.
- **Auth e BaaS:** supabase per le interazioni privilegiate con il DB e PyJWT per la validazione/decodifica sicura dei token di sessione.

3.5 Database

Il modello dati è ospitato su PostgreSQL (tramite Supabase) ed è stato rigorosamente normalizzato per garantire l'integrità referenziale del dominio energetico. La sicurezza è garantita a livello di database tramite policy **RLS (Row Level Security)**, che assicurano l'accesso ai soli dati di propria competenza.

Oltre alle normali tabelle relazionali, l'architettura sfrutta appieno l'ecosistema Supabase integrando funzioni PostgreSQL, Storage per i file multimediali (es. avatar e allegati dei ticket), Trigger Functions automatiche (es. creazione automatica del profilo in `public.users` al momento della registrazione su `auth.users`) e Viste per i dati di produzione aggregati.

Le entità relazionali principali includono:

- **users:** Estende l'entità di autenticazione di Supabase aggiungendo anagrafica completa (Nome, CF, Indirizzo), gestione del piano di abbonamento (`subscription_plan`) e flag di amministrazione globale (`is_super_admin`).
- **cer & cer_members:** Gestiscono le Comunità Energetiche e le relative iscrizioni. La tabella di giunzione traccia il ruolo dell'utente (`admin`, `member`) e lo stato di approvazione dell'iscrizione.
- **impianti:** Associa gli asset fisici di produzione fotovoltaica agli utenti, definendone coordinate geografiche, potenza nominale e stato operativo.
- **annunci & support_tickets:** Moduli per la comunicazione interna alla CER (bacheca annunci) e per il tracciamento dell'assistenza tecnica, con supporto per allegati organizzati nello Storage.

Gestione Avanzata delle Serie Storiche (Time-Series): La mole di dati generata dalla telemetria degli impianti (Gli impianti utilizzati per i test aggiornavano le misurazioni ogni 5 minuti) ha richiesto

una strategia di *Table Partitioning*. È stata definita una tabella master, **misurazioni**, che raccoglie strutturalmente le letture energetiche (produzione, consumo, stato batteria). Tuttavia, i record vengono fisicamente reindirizzati e salvati in tabelle figlie partizionate per mese (es. **misurazioni_y2025m02**). Per garantire la scalabilità e la continuità del servizio senza intervento manuale, è stato implementato un **Cron Job** a livello di database che genera automaticamente la tabella per il mese successivo con un mese di anticipo. Questo approccio garantisce query estremamente rapide sui dati correnti e facilita l'archiviazione a lungo termine. Inoltre, per ottimizzare le performance delle query e alleggerire il carico computazionale per l'elaborazione dei dati per le dashboard e grafici, sono state definite specifiche **Viste**. Queste elaborano e pre-aggregano i dati grezzi direttamente alla fonte (es. totali di produzione giornaliera o mensile), restituendo risultati pronti per essere consumati dalle dashboard.

3.6 Testing

L'approccio al collaudo del software ha adottato una **metodologia iterativa e pragmatica**, allineata alle moderne pratiche di sviluppo Agile. I casi di test sono stati implementati parallelamente allo sviluppo dei componenti, con l'obiettivo di validare il codice reale e i flussi critici dell'applicazione in scenari concreti.

Il sistema si avvale di una pipeline di **Continuous Integration (CI)** automatizzata tramite GitHub Actions. La suite di collaudo conta complessivamente **oltre 50 file di test e più di 80 test individuali**, suddivisi tra i due macro-ambienti:

- **Frontend:** I componenti React (pagine, form, tabelle) e le logiche di routing sono ampiamente testati tramite **Vitest** e *Testing Library*, garantendo che la UI reagisca correttamente ai cambi di stato e agli input dell'utente.
- **Backend:** Gli endpoint FastAPI, i calcoli analitici e le interazioni con il database sono validati tramite **Pytest**, verificando in modo rigoroso le aggregazioni sui dati (es. tramite Pandas) e la sicurezza delle rotte.

Data la vastità della test suite implementata (80+ test implementati tra Vitest e Pytest), la seguente tabella riassume per **macro-categorie** i principali flussi di validazione eseguiti automaticamente dal sistema ad ogni aggiornamento della repository:

N.	Macro Caso di Test	Test Data	Precondizioni	Dipendenze	Risultato Atteso	Esito	Note / File
Frontend (UI & Stato Integrato) - Vitest							
1	Flussi di Autenticazione: Login, Registrazione e OTP	Credenziali mock valide e invalide	Nessuna	Hook Form	I form mostrano errori (Zod) o completano il submit con successo.	OK	login-form.test.tsx, otp-form.test.tsx
2	Dashboard: Rendering widget e grafici	Mock dati energetici	Utente autenticato	Recharts	La homePage carica i widget e mostra i grafici senza crash.	OK	homePage.test.tsx
3	Gestione Impianti: Rendering asset fisici	Mock dati impianti	Utente con impianti	React Table	Popolamento corretto della plants-table e apertura dettagli.	OK	PlantsPage.test.tsx
4	Gestione CER: Tabelle membri e dettagli	Mock lista membri	Ruolo = Admin/Member	React Table	Rendering della members-table e dei dettagli utente.	OK	CerPage.test.tsx
5	Area Personale: Aggiornamento Profilo	Nuovi dati anagrafici	Utente loggato	Context API	I form di profilo e password gestiscono correttamente lo stato.	OK	userProfileForm.test.tsx
6	Supporto e Ticket: Visualizzazione e creazione	Mock dati ticket	Utente loggato	Shadcn UI	Rendering della ticket-table e gestione stato di apertura ticket.	OK	SupportPage.test.tsx
Backend (API & Analisi Dati) - Pytest							
7	Protezione Rotte: Verifica middleware Auth	GET/POST a /api/*	Token JWT assente	FastAPI	L'accesso viene negato con codice HTTP 401 Unauthorized.	OK	test_smoke.py
8	Analytics: Calcolo KPI e trend di produzione	Dati storici mock	Misurazioni nel DB	Pandas	Calcolo accurato di consumi, produzione e delta percentuale.	OK	test_analytics.py
9	Reportistica: Generazione export PDF	period = month	Storico disponibile	FPDF	L'endpoint restituisce il file binario formattato correttamente.	OK	test_reports.py
Infrastruttura (CI/CD)							
10	Continuous Deployment	Push su main	Superamento CI	Render/Vercel	Compilazione (TSC/Vite) e deploy degli ambienti completati.	OK	Log Cloud Provider

Tabella 1: Riassunto per macro-categorie della suite di test, divisa per layer architetturale.

4 FrontEnd

4.1 Descrizione FrontEnd

Il FrontEnd della Progressive Web App (PWA) Rinova è stato concepito per offrire un'esperienza utente reattiva, intuitiva e pienamente accessibile in ottica *Mobile-First*.

L'interfaccia utente non si limita all'utilizzo di librerie pre-confezionate: i componenti di base forniti da **Shadcn UI** sono stati profondamente personalizzati ed estesi tramite Tailwind CSS per creare una libreria di componenti custom (card, form, widget energetici, tabelle) coerente con la complessa identità visiva e funzionale del progetto.

4.2 Navigazione

La navigazione è separata concettualmente in due macro-aree:

- **Area Pubblica (Auth):** Gestisce l'onboarding, la registrazione, il login e il recupero delle credenziali.
- **Area Riservata:** Costituisce il core dell'applicativo e include la Dashboard energetica, lo Storico, la Gestione Impianti e la vista Comunità Energetica.

4.3 Logiche di Visualizzazione e Funzionalità

Per garantire una User Experience ottimale nella consultazione dei dati energetici, le viste sono state strutturate secondo le seguenti logiche:

- **Dashboard (Live):** I KPI principali (produzione, consumo, stato batteria) mostrano un valore *cumulativo* aggregato di tutti gli impianti dell'utente. Al contrario, per garantire un monitoraggio granulare, viene renderizzato un grafico live distinto per ogni singolo impianto registrato.
- **Storico Produzione (Analytics):** Presenta un grafico unificato. Di default la vista mostra il cumulativo storico di tutti gli asset dell'utente, con la possibilità di utilizzare filtri specifici per isolare e analizzare le performance di un singolo impianto.
- **Gestione Impianti:** Questa sezione integra funzionalità avanzate. Oltre alla visualizzazione dei dettagli tecnici e della posizione esatta su mappa (tramite **Leaflet**), l'utente può interagire con lo stato dell'asset, ad esempio segnalando un guasto o un periodo di manutenzione (sospendendo temporaneamente il calcolo dell'impianto) per poi riattivarlo a intervento concluso.

- **Comunità Energetica (CER):** L'attuale implementazione funge da *Minimum Viable Product* (MVP) per l'amministrazione interna dei membri. Consapevoli della reale complessità burocratica e in attesa di future integrazioni con la cartografia e i POD del GSE, si è scelto consapevolmente di non banalizzare il processo di adesione con automatismi fittizi, concentrando lo sforzo sulla visualizzazione del bilancio energetico e della bacheca annunci interna.

4.4 Schermate Principali

Di seguito sono illustrate le schermate principali. Per evidenziare la natura PWA e pienamente responsiva dell'applicazione, per ogni vista è proposto il confronto diretto tra il layout Desktop e quello Mobile.



Figura 1: Dashboard Live: Vista Desktop. Si notano i KPI cumulativi e i grafici di produzione rendizzati per singolo impianto.

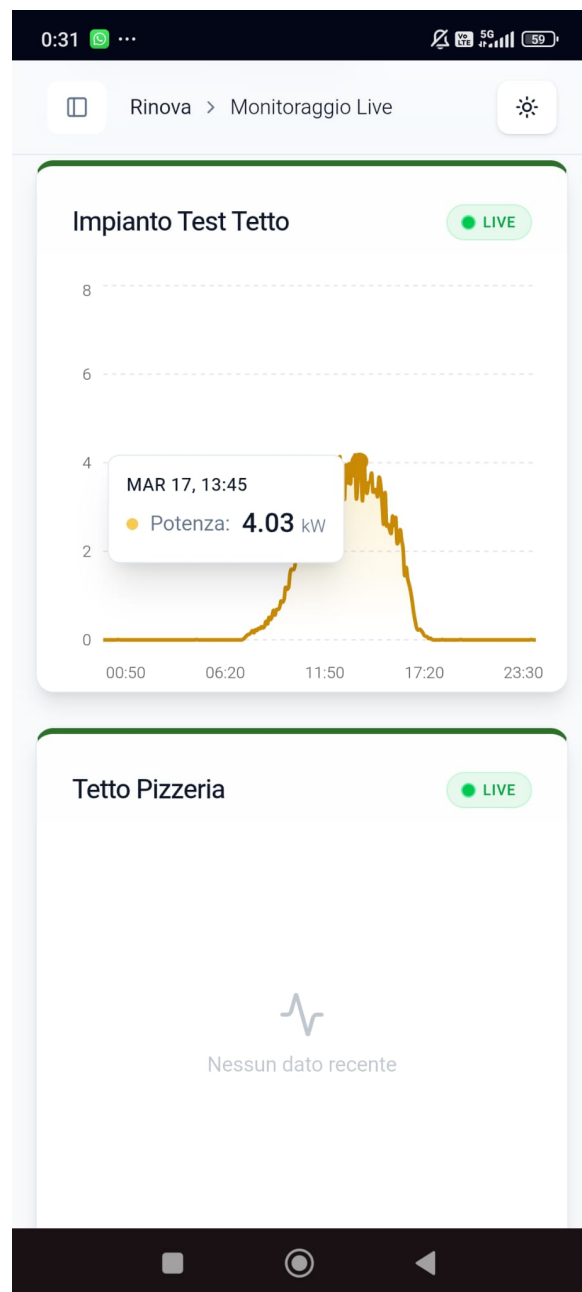


Figura 2: Dashboard Live: Vista Mobile. Si notano i KPI cumulativi e i grafici di produzione rende-
rizzati per singolo impianto.

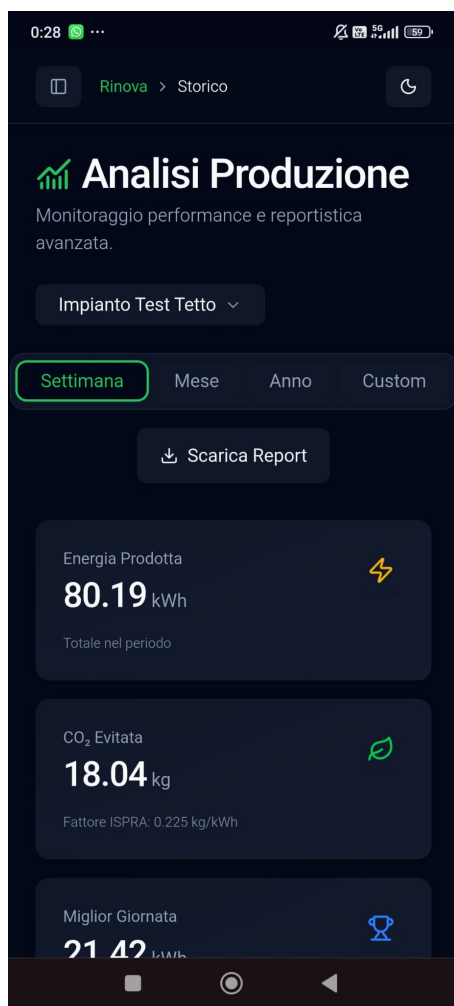
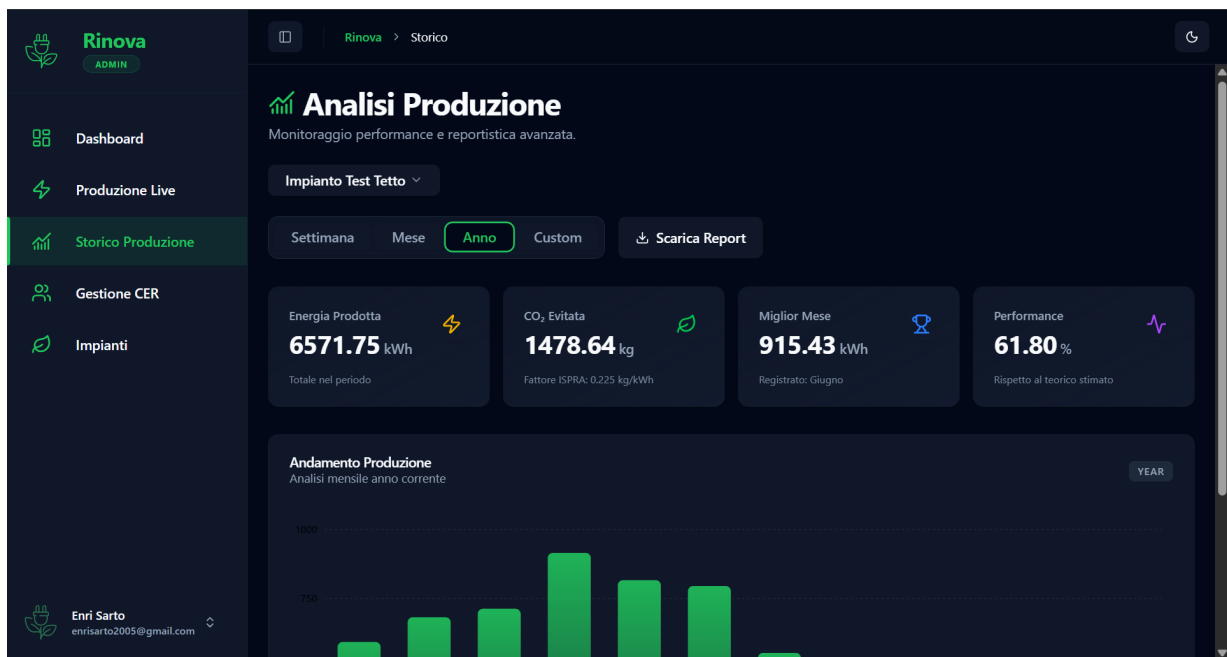


Figura 3: Storico Produzione: Vista Desktop (sopra) e Viste Mobile (sotto). Grafico unificato con possibilità di filtraggio per singolo asset.

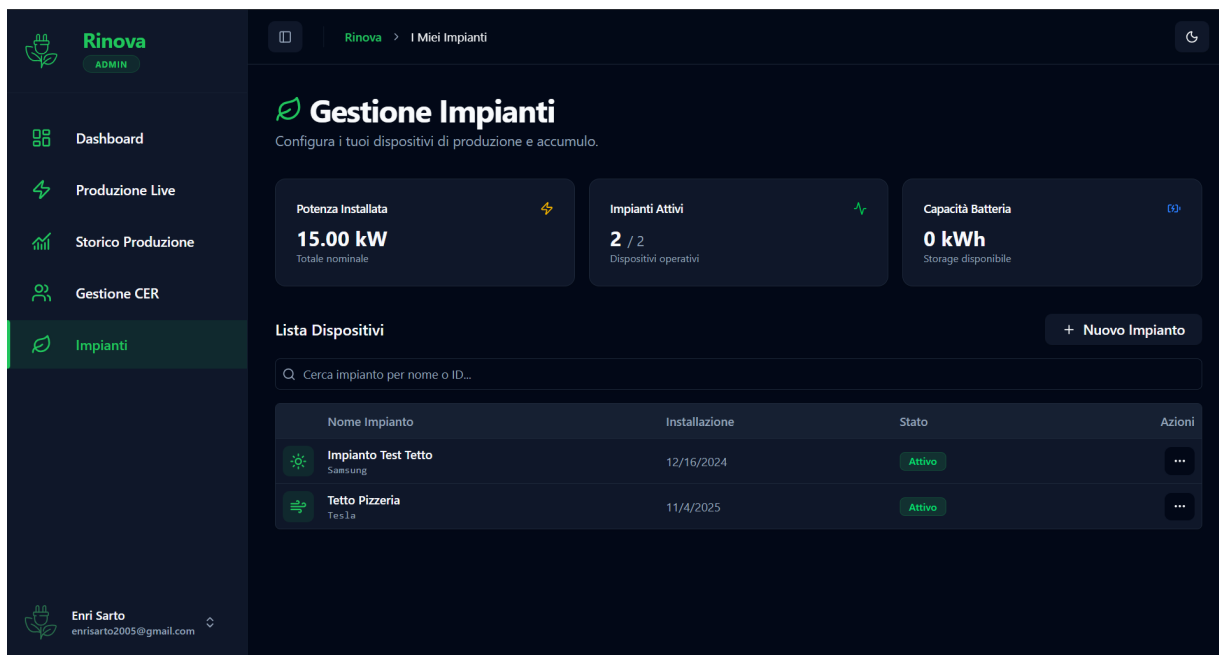


Figura 4: Gestione Impianti: Vista Desktop (sopra) con una visualizzazione tabellare per il riepilogo della potenza installata, dello stato operativo e Vista Dettaglio Mobile (sotto), con dettagli tecnici dell'impianto e integrazione mappa Leaflet per la posizione. Funzionalità di segnalazione guasti e manutenzioni sono accessibili in layout Desktop e Mobile tramite menù a tendina.

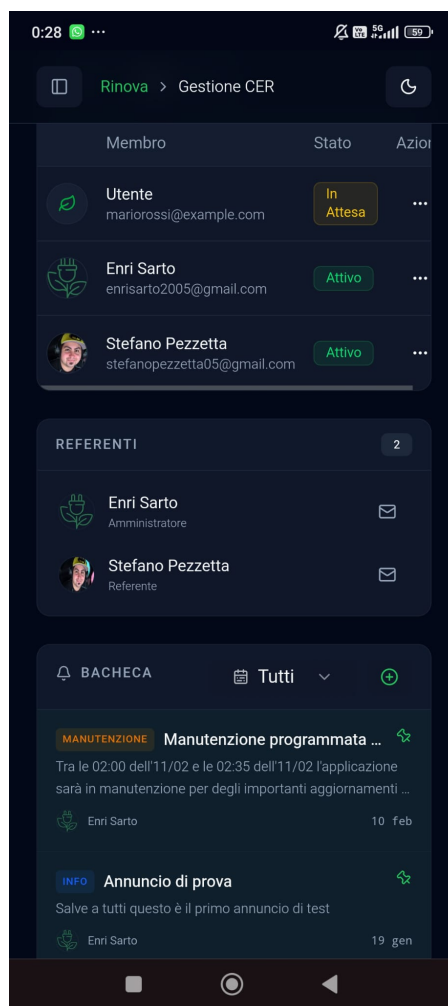
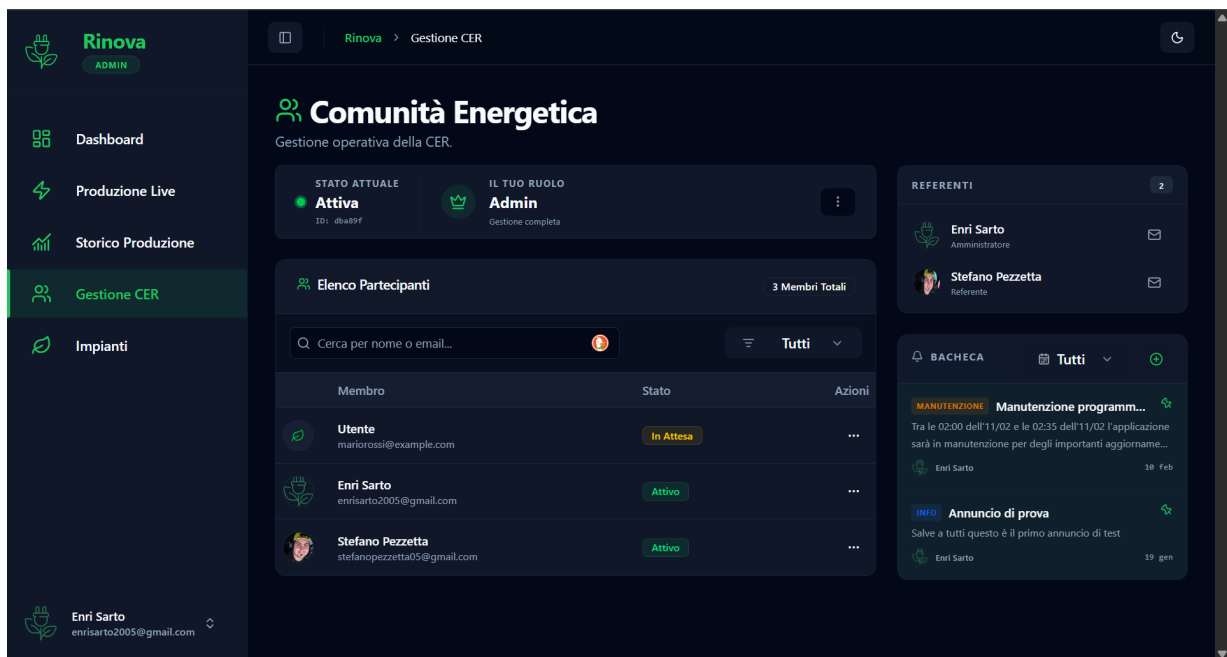


Figura 5: Gestione CER: Vista Desktop (sopra) e Vista Mobile (sotto). MVP per l'amministrazione della bacheca e dei ruoli della comunità, ottimizzato per entrambi i dispositivi.

5 Deployment

L'applicazione Rinova adotta una strategia di deployment distribuita, separando fisicamente l'hosting del frontend da quello del backend per ottimizzare le risorse e aggirare le limitazioni delle piattaforme cloud gratuite.

Nello specifico, il sistema è ospitato sui seguenti provider:

- **FrontEnd (PWA):** Ospitato su **Vercel**. <https://rinovaenergy.vercel.app/>
- **BackEnd (FastAPI):** Ospitato su **Render**. <https://rinova-energy-api.onrender.com>
- **Database & Auth:** Gestiti tramite l'infrastruttura cloud di **Supabase**.

Motivazioni Architetture del Deployment Separato: La scelta di mantenere i due layer su piattaforme diverse è dettata dalle severe limitazioni degli ambienti *Serverless* per Python offerti da provider come Vercel. L'elaborazione analitica dei dati energetici e la generazione procedurale dei report PDF avrebbero saturato rapidamente i limiti di computazione (timeout delle funzioni e limiti di memoria) previsti dai piani gratuiti. Mantenendo il backend su un'istanza dedicata (Render), il server ha accesso continuo alle risorse necessarie per completare i task più onerosi.

Gestione dello Standby (Render): L'istanza backend su Render (tier gratuito) entra in stato di *sleep* dopo 15 minuti di inattività, comportando un avvio ritardato (*cold start*) alla prima richiesta successiva. Tuttavia, grazie a un semplice script di *keep-alive* (es. un cron job esterno che effettua il ping periodico dell'endpoint di health check /), è possibile mantenere l'istanza attiva coprendo l'intero mese senza superare il tetto massimo di ore di calcolo fornite dalla piattaforma.

5.1 Accesso all'Applicativo

Il sistema è configurato tramite *Continuous Deployment (CD)*: ogni operazione di push sul branch principale main innesca automaticamente la fase di building e il rilascio in produzione sui rispettivi provider.

Per testare le funzionalità dell'applicazione e valutare la logica di *Role-Based Access Control (RBAC)* in sede d'esame, sono stati predisposti i seguenti account di test:

Utente Membro Standard (Accesso alla dashboard e ai propri impianti)

- Username: `test_utente@rinova.app`
- Password: `7es7U7entee!R`

Utente Amministratore CER (Accesso alle funzionalità di gestione comunità)

- Username: `test_admin@rinova.app`
- Password: `7es7Adm!nn!Rin`

5.2 Contatti Assistenza

Per eventuali problematiche riscontrate durante il testing dell'applicazione o l'accesso al deployment, è possibile contattare i referenti di progetto:

- `enrisarto2005@gmail.com` (Sviluppatore Full-Stack / Amministratore)
- `stedanopezzetta05@gmail.com` (Project Manager / Documentazione)

Galleria Interfacce Secondarie

Di seguito si riportano ulteriori viste che compongono l'applicativo, a dimostrazione della coerenza visiva, dell'estensione del lavoro svolto sulla User Interface e della cura riposta nei flussi di interazione secondari.

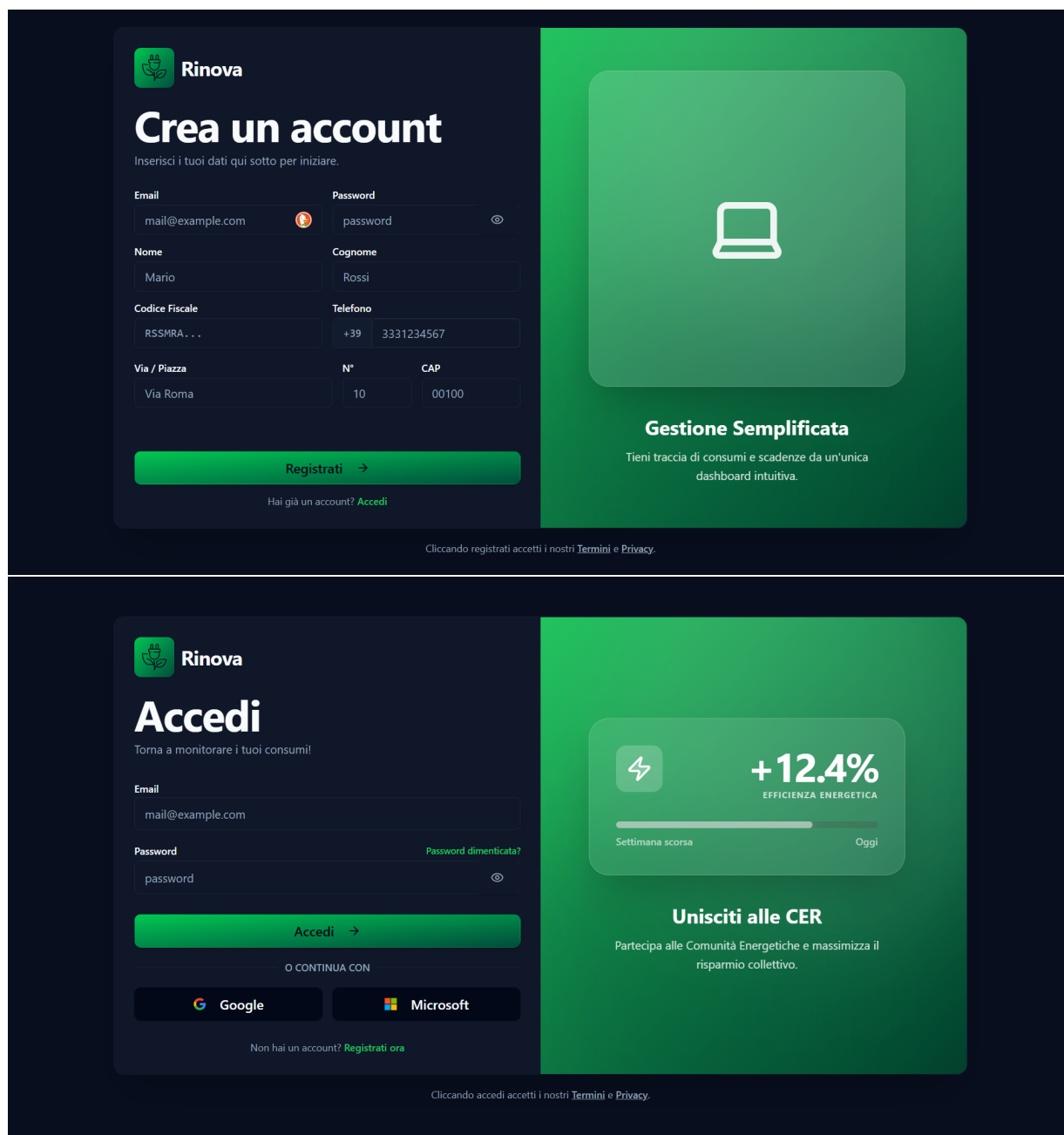


Figura 6: Flusso di Autenticazione: Il layout pubblico garantisce un onboarding fluido e sicuro, rispettando i criteri di accessibilità.

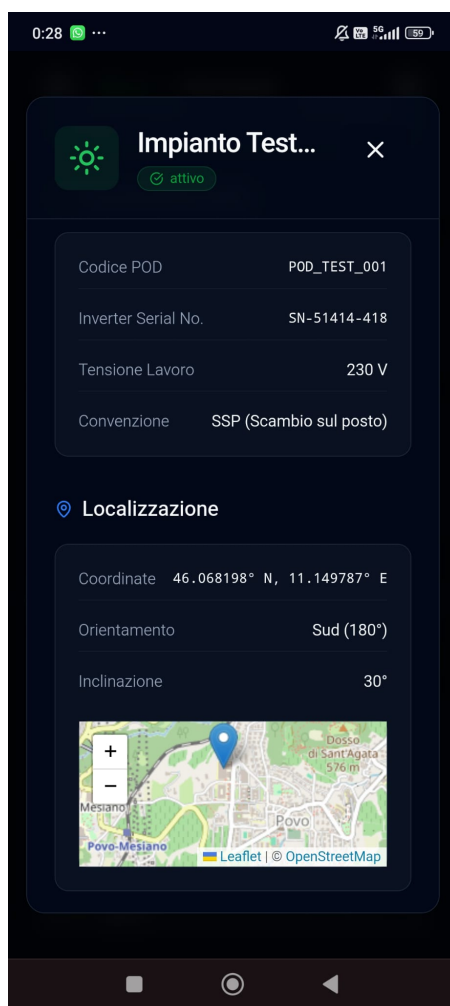
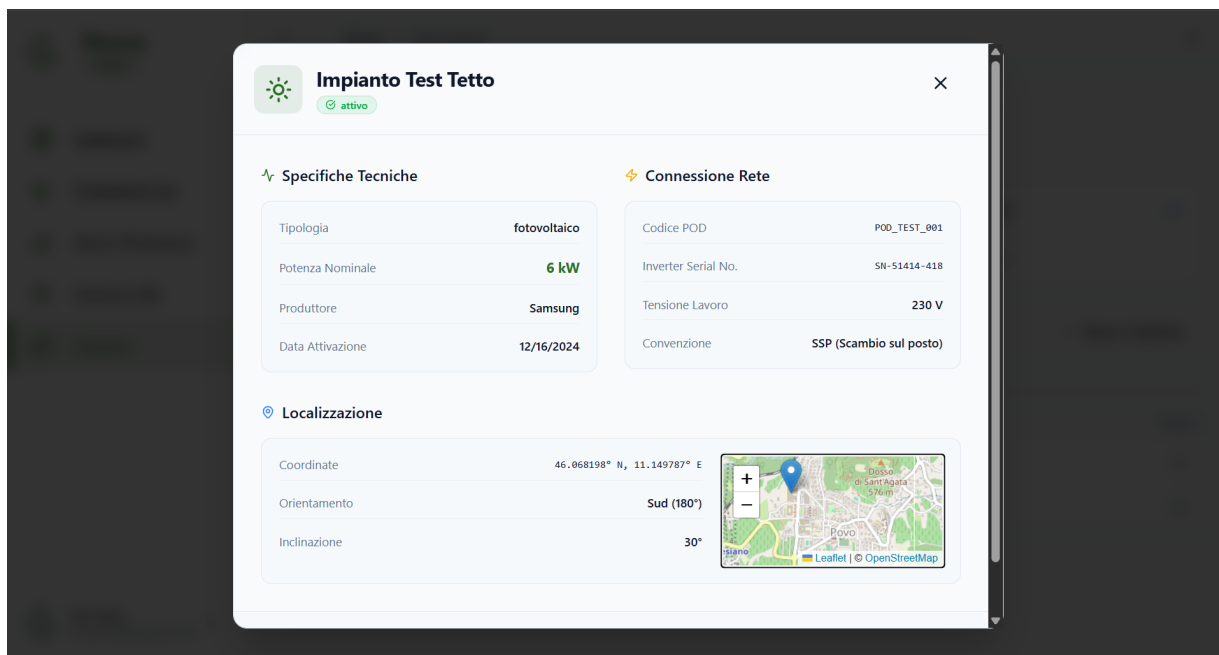


Figura 7: Dettaglio Impianto: Vista Desktop (sopra) e Vista Mobile (sotto). Integrazione mappa Leaflet e funzionalità di segnalazione guasti in layout Desktop e Mobile.

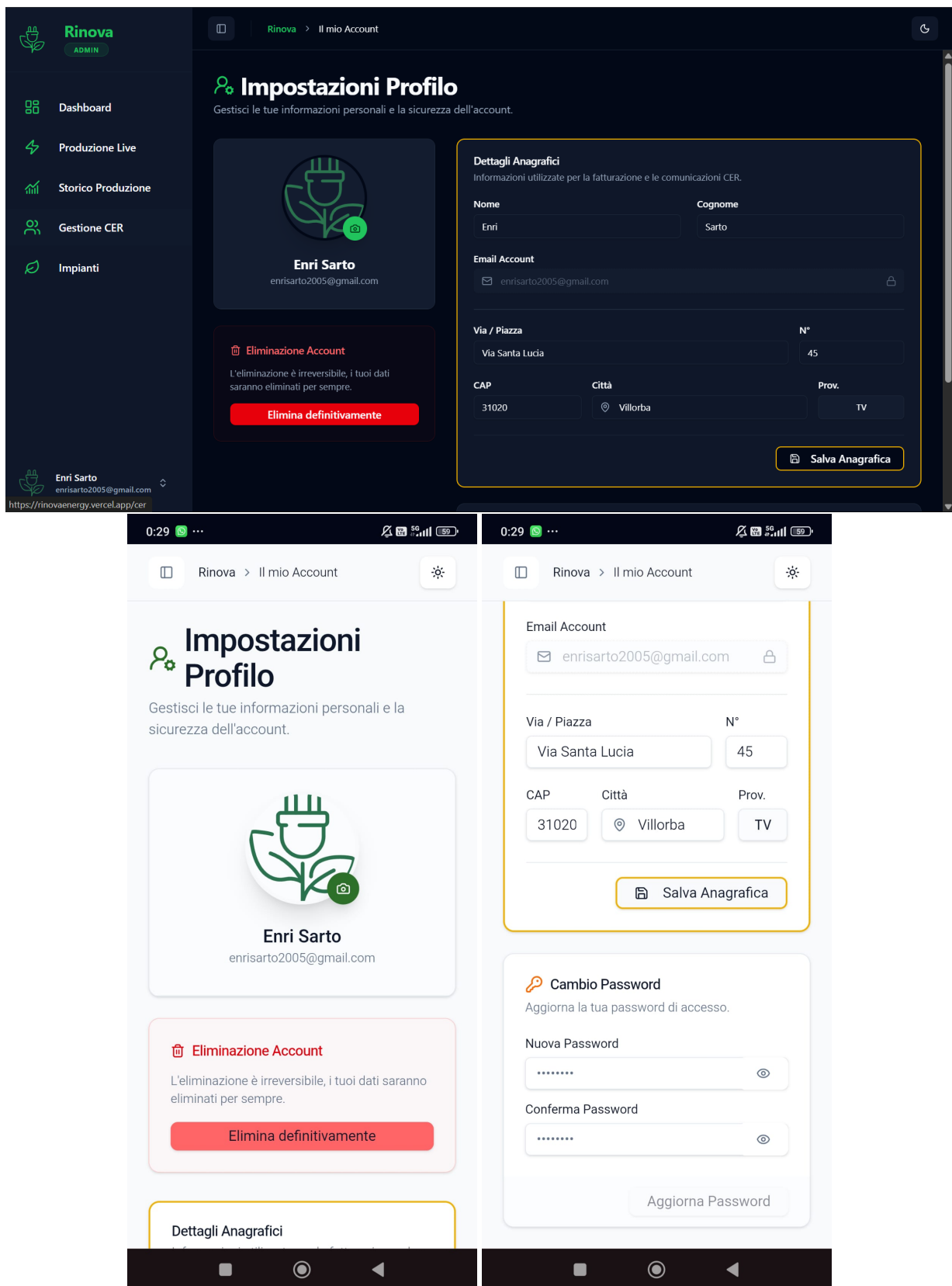


Figura 8: Area Personale: Gestione del profilo utente e aggiornamento in sicurezza delle credenziali tramite componenti form validati con Zod.

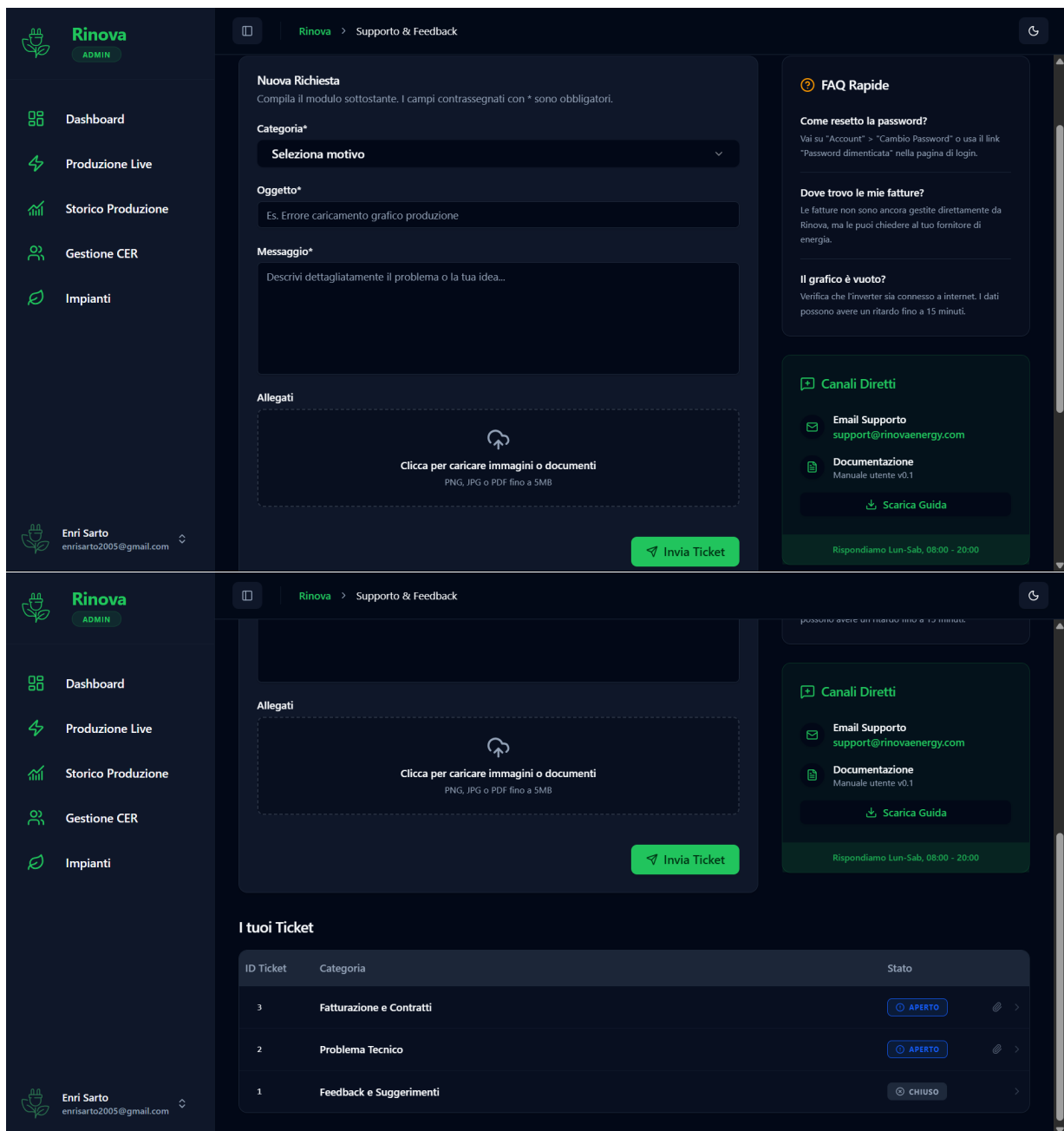


Figura 9: Supporto Tecnico: Interfaccia per la visualizzazione e l'apertura di nuovi ticket di assistenza integrata all'interno della dashboard. Le Frequently Asked Questions sono disposte in una colonna a sinistra, oltre agli orari per il contatto diretto via email.

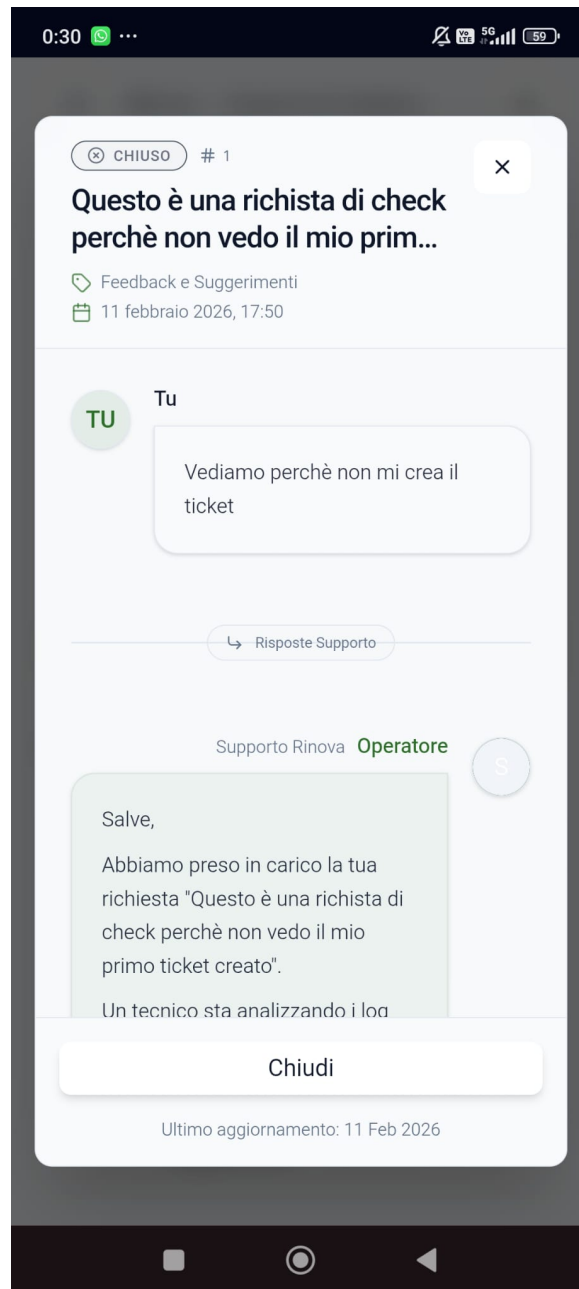


Figura 10: Dettaglio Ticket: Interfaccia per visualizzare i ticket creati dall'utente, il loro stato e le risposte degli operatori

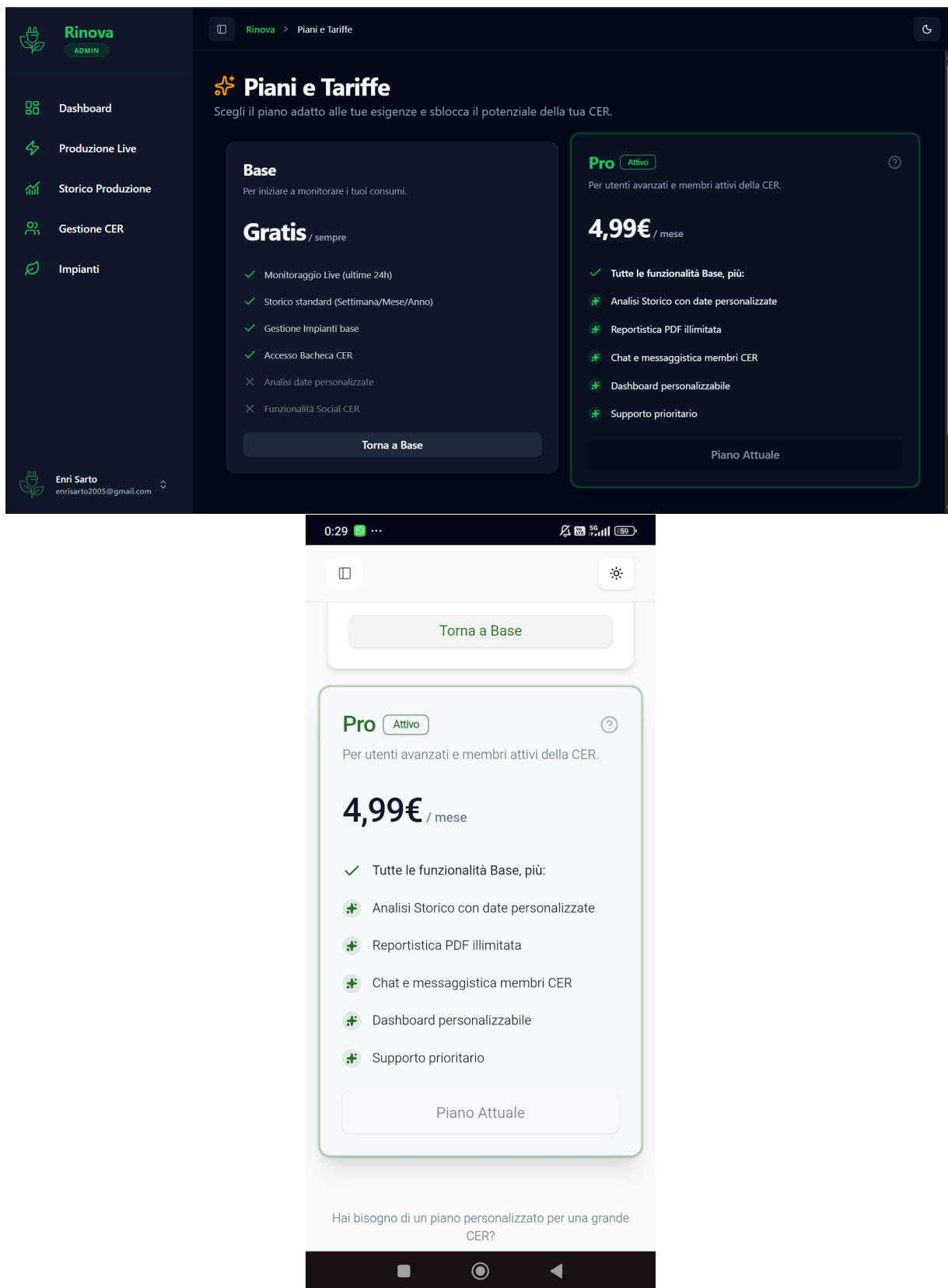


Figura 11: Piani Base e Pro: Interfaccia per la selezione del piano di abbonamento all'applicazione, la funzionalità di Pagamento non è ancora stata implementata e sarà disponibile solo in versioni successive dell'applicazione, le funzionalità del piano Pro rimangono accessibili ai soli tester e developers.

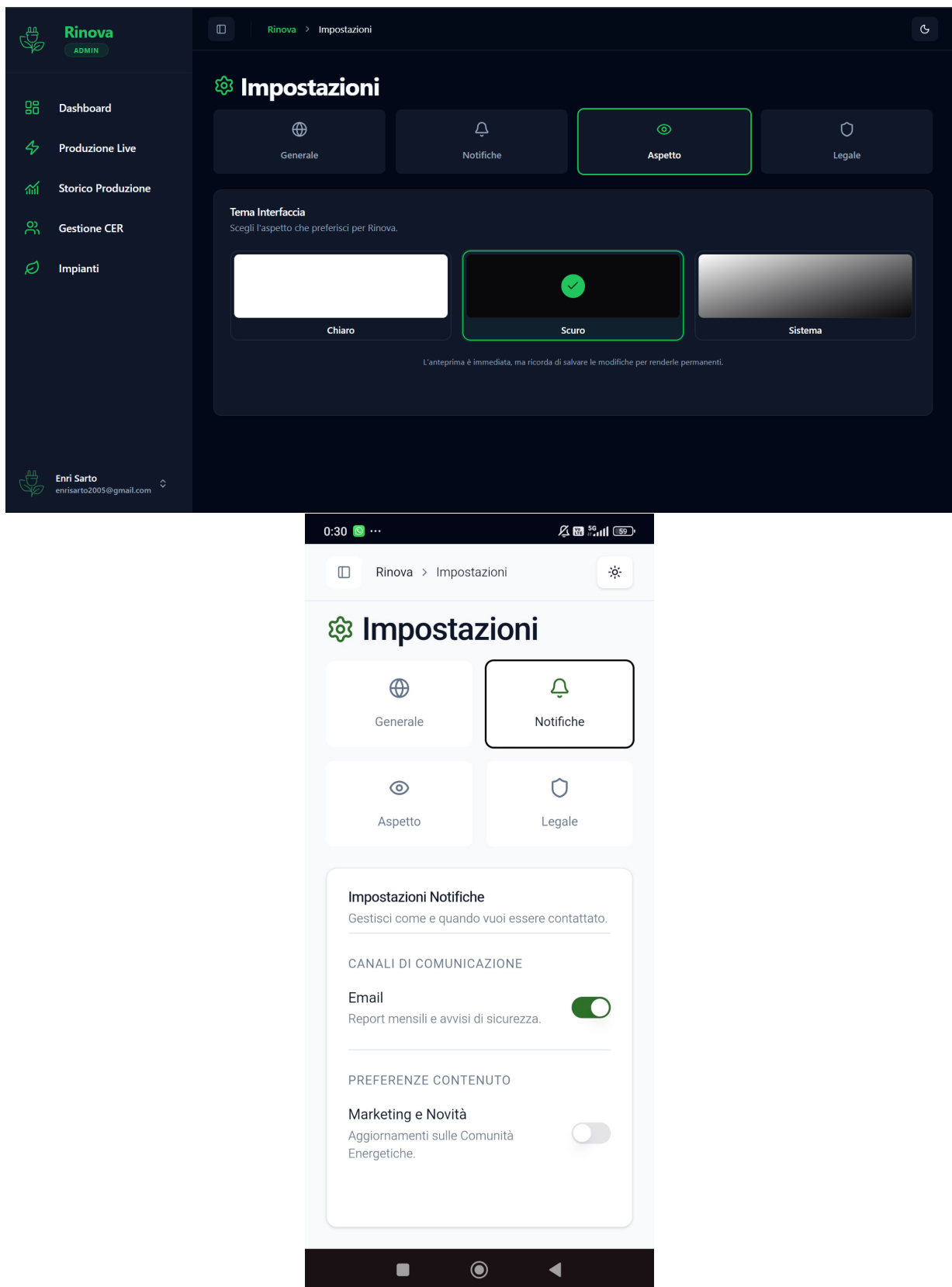


Figura 12: Impostazioni: Interfaccia per la personalizzazione dell'aspetto e delle preferenze all'interno dell'applicazione. Nella tab "Legale" sono consultabili anche i **ToS(Terms of Service)** e la **Privacy Policy**.